Enterprise JavaBeans 3.2

Sample code updated to ee10:
https://github.com/bdlink/oreilly-ejb-6thedition-book-examples.git
use ee10 branch

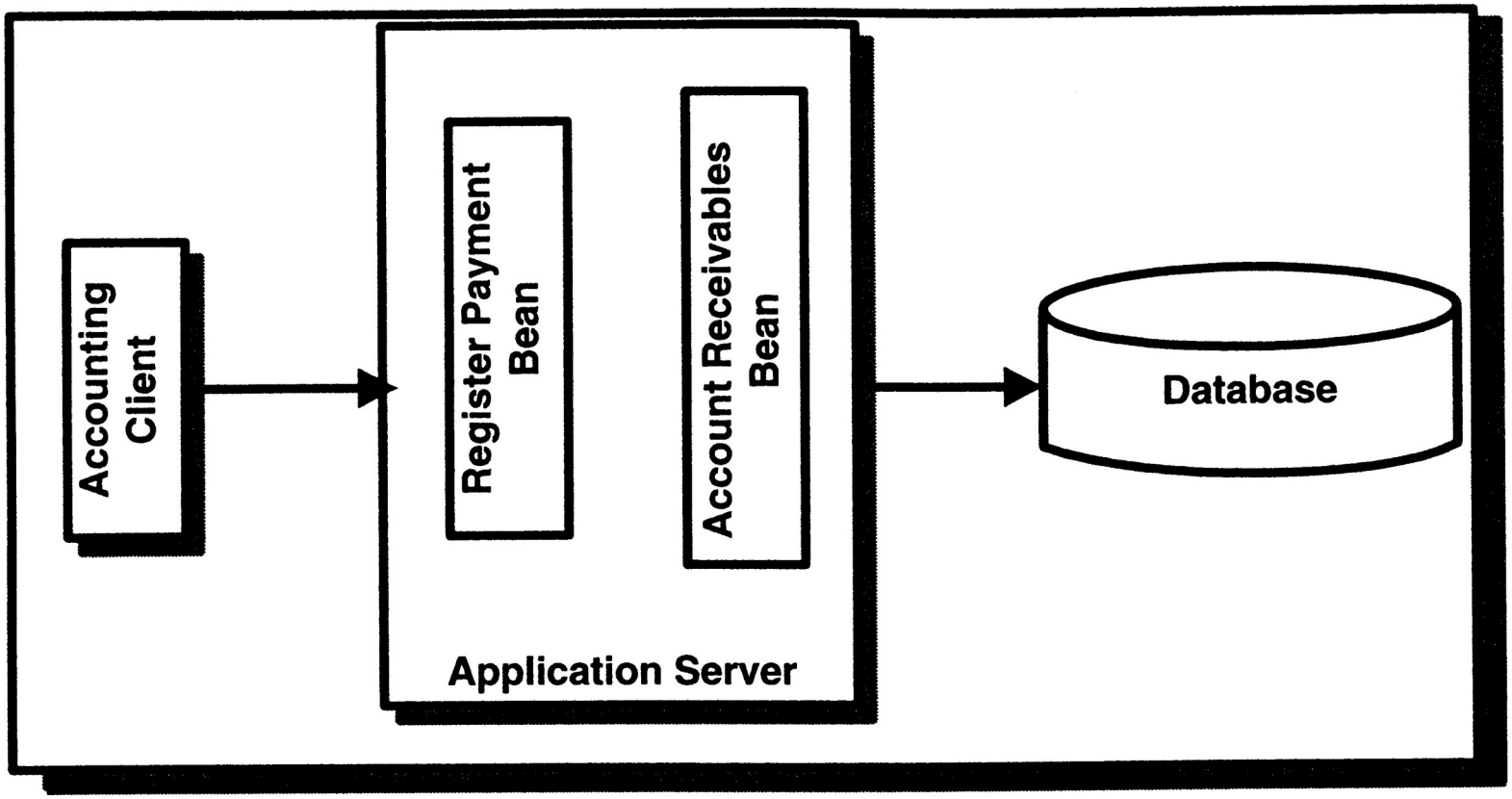
Introduction

Chapter 1

Enterprise JavaBeans

- EJBs are components
  - That live on a server
- EJB is a standard
  - Allows distributed server components to be designed that plug together easily
  - Allows multiple vendors to supply compliant application servers
  - Similar in concept to JavaBeans
- Programmers don't have to deal with
  - Multithreading, transactions, connection pooling
  - All handled by the EJB server

Example EJB Application



EJB Goals

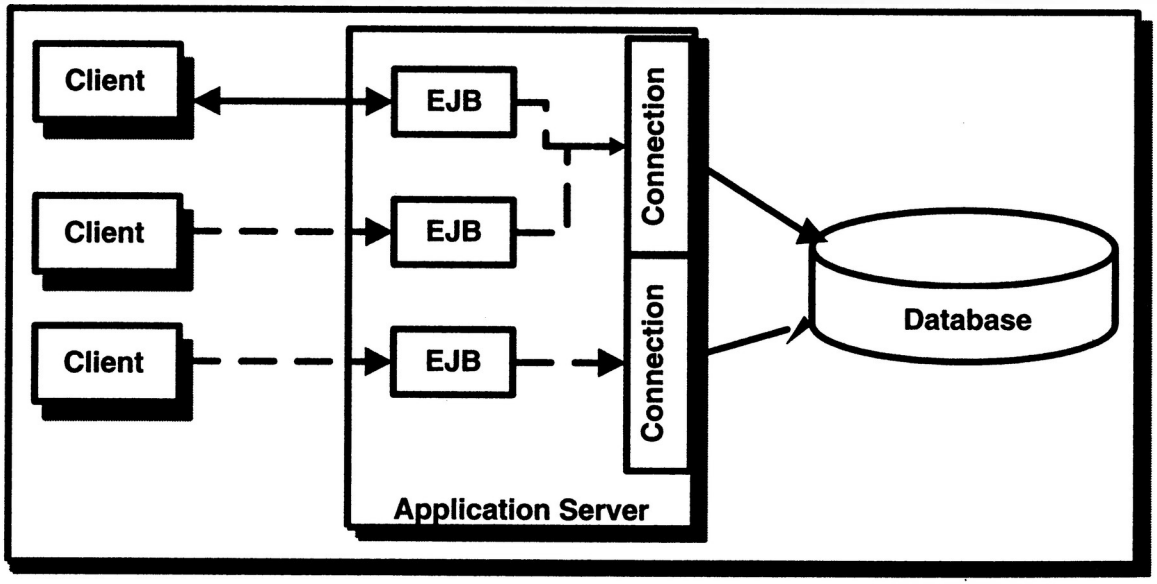
- Clearly define how application is partitioned
  - Client – EJBs – Servers
- Improve scalability and performance over ad-hoc technologies
- Increase reliability by encapsulating EJB
- Centralize resources for easier management
- Promote reuse
- Support different types of clients with one component technology

EJB Containers

- EJBs are Java objects that have a specific interface
- They are run by a server using an EJB container to interface the EJB with the outside world
- Container provides services to the EJB
  - Such as connection pooling
- Clients talk to the EJB through a container interface

Resource Pooling

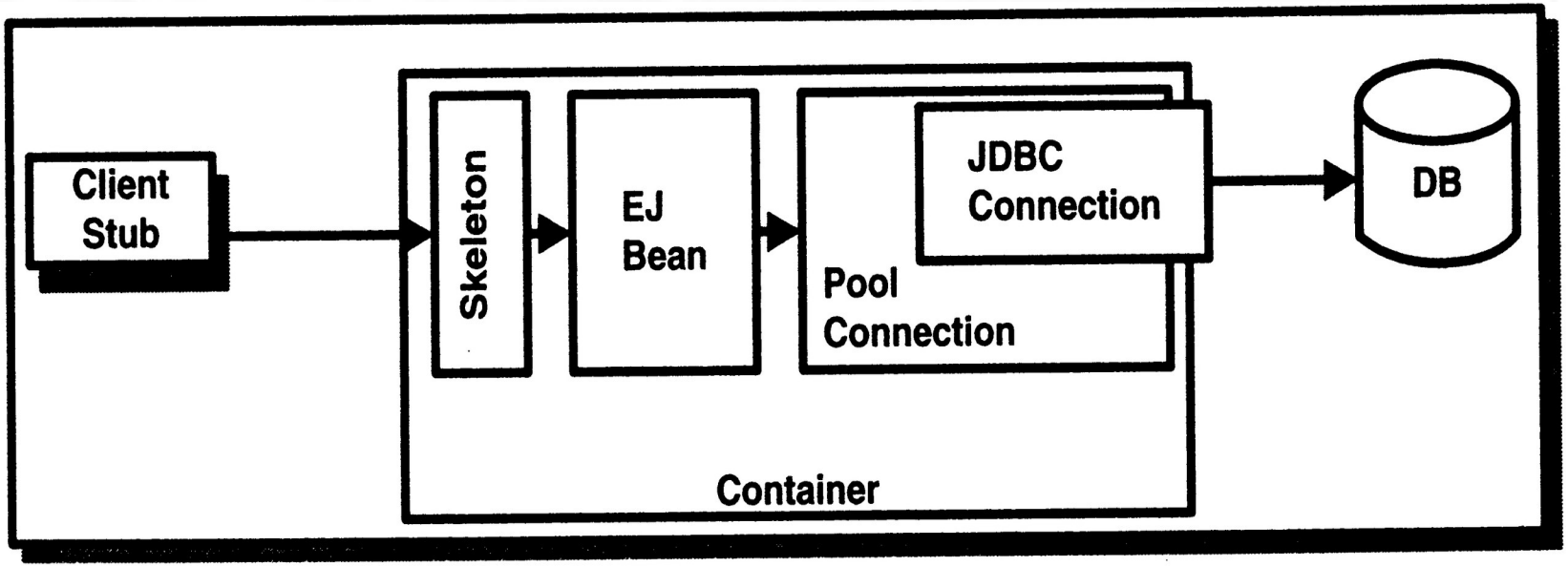
- JDBC requires connection to database
  - Long time to establish and consumes resources
- If not all EJB instances need the connection at same time, connections can be shared:



How EJBs Fit Inside Containers

- Clients don't talk directly to EJB
- Clients talk to container-generated skeleton
  - EJB Object
- Container manages pools, resources, transactions
- EJB programmer concentrates on application logic
- EJB gets easy database access, transaction management, location services, well-defined life cycle

EJB in Container



- Implemented using RMI or CORBA (under the covers)

Asynchronous Messaging

- Another type of EJB supports asynchronous messaging
  - Using Java Message Service
- Asynchronous messaging allows a client to send a request and have it processed at a later time
  - Such as sending an order of books to the shipping department
- Messages can be queued and processed when resources come available

Web Services

- An XML-based way of discovering and accessing distributed components
- Platform-independent but inefficient
- Uses SOAP and WSDL to exchange information as XML documents
  - SOAP provides the calling parameters and return result
  - Web Service Description Language (WSDL) describes the interface to the service
- EJBs can allow their methods to be exposed as web services

Component Types

Chapter 2

Using Business Objects

- Since EJB 3.0, Enterprise JavaBeans created from normal Java classes
  - Called POJOs (Plain Old Java Objects)
  - Created to model the business logic
- POJOs become EJBs when
  - Assembled and packaged
  - Deployed to an EJB container
  - Accessed via the container
- Container equips POJO with EJB services
  - Exposes behaviour as Session Beans, Message-Driven Beans, Entity Beans

Three Kinds of EJBs

- Session Beans
  - Provide application services to client
  - Represent *business logic and services*
  - May be stateless, stateful (have client sessions) or singleton
- Message-Driven Beans
  - Respond to JMS messages – client ignorant of bean details
- Entity Beans (sort of)
  - Represent *business objects* in the application
  - Can allow container to make persistent or do it themselves (perhaps using JDBC)

Session Beans

- May implement `SessionBean` interface
- Represent big chunks of related *application functionality*
  - Perform *actions* on behalf of client
- Interacts with a *single client* at a time
- Can handle transactions
- Not intended to represent persistent data
  - Stateful ones should be serializable so bean can be written to file and restored
- Relatively short lived – from the client viewpoint

Message-Driven Beans

- May implement `MessageDrivenBean` interface
- Responds to JMS messages from client
  - Can be selective and only process messages on a specific Topic or Queue
- Reduces coupling between Client and bean
  - Client does not need to know about EJB model
- Server listens for messages to send to bean
  - Bean can be activated when there are messages to process

Entity Beans

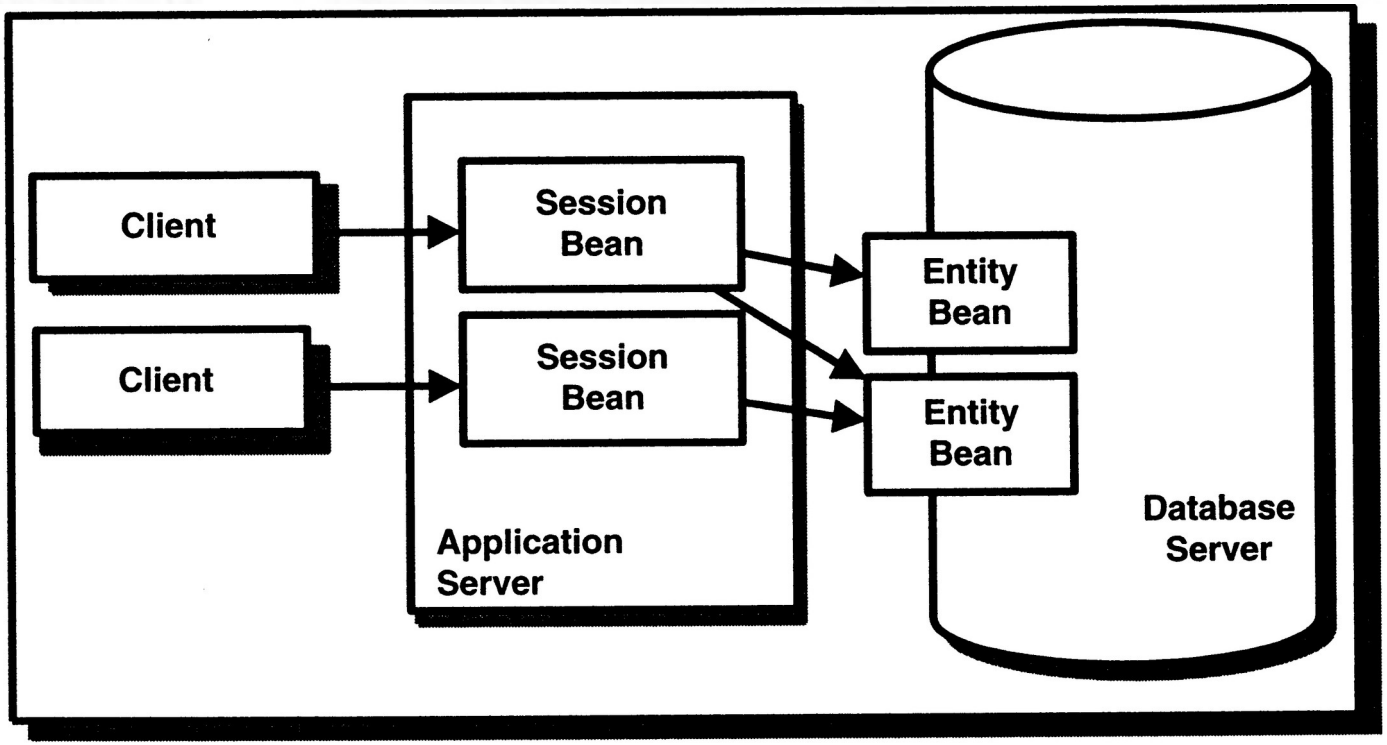
- May implement `EntityBean` interface
- Represent *application objects* (entities)
- Have persistent state
  - Either managed by the container or the bean
    - Container-managed means programmer doesn't have to worry about it, deployer defines persistence
    - Bean-managed may use files or databases to store their state
  - Can think of them as a row in a database
- State changes occur in a transaction (default)
- Can be accessed by *multiple clients* at a time

JPA Persistence Beans

- EJB 3.0 and 3.1 two types of entity beans
  - Legacy entity beans which *are* EJBs
    - Not used on new systems
  - Entity beans satisfying the Java Persistence Architecture (JPA) – *not* really EJBs
- JPA or Jakarta Persistence Beans are POJOs
  - Can be managed by container
    - Controlled by jakarta.persistence.EntityManager
  - If unmanaged, behave as any other Java Object

Multi-Tier Enterprise Beans

- Example with application server and database server both hosting EJBs



Model Taskflow: Session Beans

- Entity beans represent data, session beans represent process or actions
- Session beans are a good place to put business logic
- Session beans will use entity beans
  - Typically using a local interface for efficiency
- Clients will usually only call session or message-driven beans
  - Seldom entity beans directly
  - Unmanaged entity beans can be used as Data Transfer Objects (also called Value Objects)

When to Use EJBs

- On server end of multi-tier system
- Can centralize business processing rules
- Can hide application implementation to protect intellectual property
- Allows more centralized security
- Can hide details of legacy systems and provide better interfaces
  - Entity beans can hide database changes
  - Entity beans can translate database details into more Java-friendly terms
- Can reduce processing required by client

Summary

- EJBs are a powerful component-building tool for enterprise-level applications
- First half of the course will cover the essentials
  - What you will need to use EJBs on the project
- Second half of course covers additional material you may find useful on the project
- EJBs have been greatly simplified in the 3.0 and later specifications!